



实验报告

课程 嵌入式系统及应用

题目 GPIO 输入与 EXTI 外中断实验

院系名称_____

专业_____班级_____

学生姓名_____学号_____

任课教师_____

时 间_____

一、实验设备与软件环境

设备:

- 1 PC 电脑, 至少需要 2 个 Type-A 的 USB 插口;
- 2 ALIENTEK MiniSTM32 实验套件;
- 3 STLINK/SWD 调试下载器或 USB 串口下载线。

环境:

macOS 26.3; CLion 2025.3; CMake、OpenOCD、arm-none-eabi-gdb 工具链。

二、实验任务

程序查询方式, 主程序中连续不停地查询按键, 并根据按键状态完成相应处理:

- 1 按给出的演示程序验证: KEY0 按下发光二极管 DS0 亮, KEY0 弹起来 DS0 灭;
- 2 查询方式—自行编写程序: KEY1 按下发光二极管 DS1 亮, KEY1 弹起来 DS1 灭; WK_UP 按下发光二极管 DS0 亮、DS1 灭, WK_UP 弹起来 DS0 灭、DS1 亮;
- 3 查询方式—自行编写程序: KEY0 按第一次发光二极管 DS0 亮, KEY0 按第二次发光二极管 DS0 灭, 以此类推; KEY1 按第一次发光二极管 DS1 亮, KEY1 按第二次发光二极管 DS1 灭, 以此类推; WK_UP 按第一次发光二极管 DS0、DS1 亮, WK_UP 按第二次发光二极管 DS0、DS1 灭, 以此类推;
- 4 查询方式—解决按键延时去抖动;

中断方式, 初始化外中断, 中断检测按键并根据按键状态完成相应处理, 主程序可完成其它任务:

- 1 中断方式—按给出的演示程序验证: WK_UP 第一次按下发光二极管 DS0 和 DS1 同时亮, WK_UP 第二次按下 DS0 和 DS1 灭;
- 2 中断方式—使用外中断自行编写程序: 第一次按下 KEY0 发光二极管 DS0 亮, 第二次按下 KEY0 发光二极管 DS0 灭, 以此类推; KEY1 按第一次发光二极管 DS1 亮, KEY1 按第二次发光二极管 DS1 灭;
- 3 中断方式—考虑 EXTI9_5_IRQHandler()和 EXTI15_10_IRQHandler()中断中有多个线中断判断, 重新编写程序。

三、主要程序分析

3.1 程序总体结构

本实验实际工程按任务分为 Q1 ~ Q7 七个子工程。每个子工程都保留相同的底层 board.c/board.h 抽象层, 差别主要体现在 main.c 的轮询逻辑以及 stm32f10x_it.c 中的外部中断服务函数。

文件/模块	主要作用
board.h	声明 LED、按键、延时和 EXTI 初始化接口。

board.c	初始化 PA8、PD2、PC5、PA15、PA0；封装 LED 点亮/熄灭/翻转、按键读取、SysTick 毫秒延时和 EXTI 线初始化。
Q1 ~ Q4/main.c	完成查询方式控制：电平跟随、状态翻转、按键去抖。
Q5 ~ Q7/main.c	初始化外部中断后主循环空转，按键响应交给中断服务函数。
stm32f10x_it.c	实现 EXTI0、EXTI9_5、EXTI15_10 中断处理及多线共用中断源判断。

3.2 实验涉及的 GPIO 与外设连接关系

结合开发板资料与实际工程代码，本实验使用 2 个 LED 输出端和 3 个按键输入端。DS0、DS1 均为低电平点亮；KEY0、KEY1 为上拉输入，按下为低电平；WK_UP 为下拉输入，按下为高电平。

引脚	对应器件	实际工程配置	有效电平/作用
PA8	DS0 红色 LED	GPIO_Mode_Out_PP, 2MHz	输出低电平点亮，输出高电平熄灭
PD2	DS1 绿色 LED	GPIO_Mode_Out_PP, 2MHz	输出低电平点亮，输出高电平熄灭
PC5	KEY0	GPIO_Mode_IPU	按下为低电平；外中断映射到 EXTI5
PA15	KEY1	GPIO_Mode_IPU	按下为低电平；外中断映射到 EXTI15
PA0	WK_UP	GPIO_Mode_IPD	按下为高电平；外中断映射到 EXTI0

PA15 默认与 JTAG 的 JTDI 功能复用。工程在 board_gpio_init() 中调用 GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE)，关闭 JTAG、保留 SWD，从而释放 PA15 作为 KEY1 输入。

3.3 查询方式程序实现

Q1 和 Q2 属于电平跟随型查询：主循环每 5ms 读取一次按键电平，并立即把当前按键状态映射到 LED。Q1 中 KEY0 按下时 DS0 点亮，松开时 DS0 熄灭；Q2 中 WK_UP 优先级最高，按下时 DS0 亮、DS1 灭，未按下时 DS0 灭、DS1 跟随 KEY1。

Q3 属于按键边沿检测型查询：程序记录 key0_last、key1_last、wkup_last 三个上一拍状态，只有检测到“未按下→按下”的瞬间才调用 toggle 函数，因此长按不会持续翻转。

Q4 在 Q3 的基础上加入 20ms 去抖。程序先判断按键有效，再延时 20ms 二次确认；确认后等待按键释放，再延时 20ms 过滤松手抖动，最后才返回一次稳定单击事件。

3.4 外部中断方式程序实现

Q5 使用 WK_UP 对应的 EXTI0。board_key_exti_wkup_init() 将 PA0 映射到 EXTI_Line0，触发方式为 EXTI_Trigger_Rising。中断服务函数 EXTI0_IRQHandler() 判断 EXTI_Line0 挂起后，调用 board_led_all_toggle()，实现 WK_UP 每按一次 DS0 和 DS1 同

时翻转。

Q6 使用 KEY0 和 KEY1 对应的外部中断。KEY0 位于 PC5，对应 EXTI5，并由 EXTI9_5_IRQHandler() 处理；KEY1 位于 PA15，对应 EXTI15，并由 EXTI15_10_IRQHandler() 处理。二者均设置为下降沿触发，分别翻转 DS0 和 DS1。

Q7 针对 EXTI9_5 和 EXTI15_10 这种“多个中断线共用一个中断入口”的情况重新编写程序。中断入口不再只判断一条线，而是遍历该组所有线：EXTI9_5 遍历 5~9，EXTI15_10 遍历 10~15；若某一线的挂起位有效，就执行对应动作并清除该线挂起标志。

四、实验连线

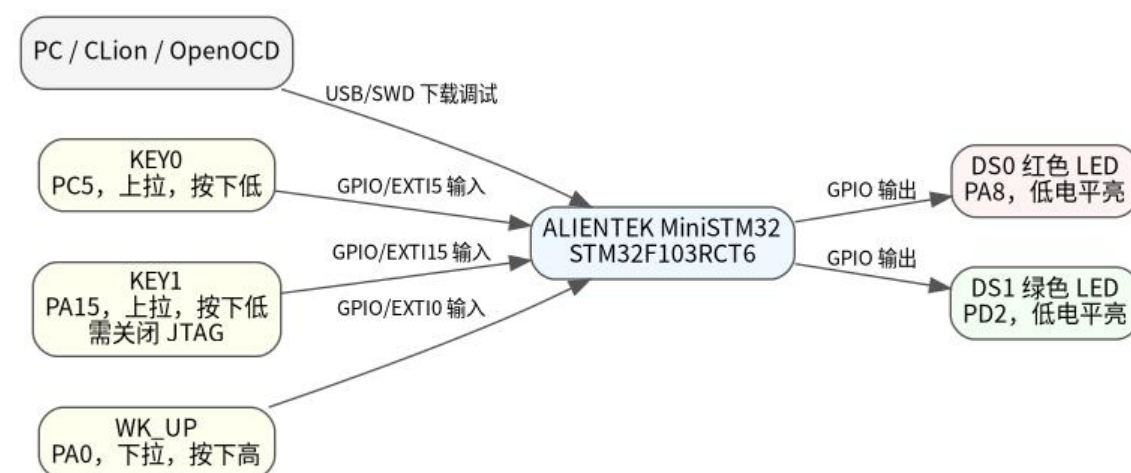


图 4-1 GPIO 输入与 EXTI 外中断实验连线/连接关系示意图

本实验全部使用 MiniSTM32 开发板板载按键和板载 LED，不需要外接 miniIO 扩展板，也不需要额外杜邦线连接 LED 或按键。实际连接重点是保证开发板供电正常，并通过 USB/SWD 完成下载调试。

连接/资源	连接位置	说明
PC 与开发板	USB 串口或 STLINK/SWD	用于下载程序、复位和调试。
DS0	板载 PA8	程序通过 PA8 控制，低电平点亮。
DS1	板载 PD2	程序通过 PD2 控制，低电平点亮。
KEY0	板载 PC5	上拉输入，按下为低；中断方式使用 EXTI5。
KEY1	板载 PA15	上拉输入，按下为低；中断方式使用 EXTI15，需关闭 JTAG。
WK_UP	板载 PA0	下拉输入，按下为高；中断方式使用 EXTI0。

LED 端口采用推挽输出，按键端口采用内部上拉或下拉输入。外部中断方式下，除 GPIO 输入本身外，还需要配置 AFIO 的端口映射、EXTI 触发边沿和 NVIC 中断通道。

五、实验步骤

- 1 阅读实验报告模板，明确本次实验需完成查询方式 Q1~Q4 和中断方式 Q5~Q7；
- 2 阅读实际工程代码，确认板载 DS0、DS1、KEY0、KEY1、WK_UP 的 GPIO 引脚与有效电平；
- 3 在 CLion/CMake 工程中分别打开 Q1~Q7，检查 main.c、board.c 和 stm32f10x_it.c 的实现；
- 4 编译并下载各子工程到 MiniSTM32 开发板，按任务逐项观察 LED 现象；
- 5 针对 KEY1、LED 极性、按键抖动和外部中断共用入口等问题修改并验证程序；
- 6 整理主要程序、流程图、调试现象和思考题，完成实验报告。

六、程序流程图



图 6-1 查询方式 Q1/Q2 电平跟随流程图

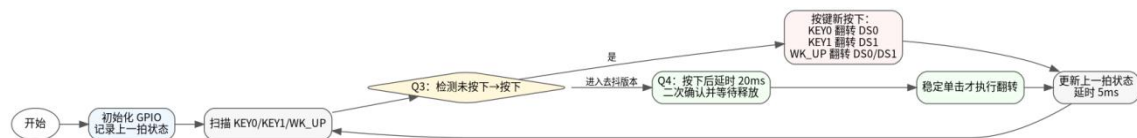


图 6-2 查询方式 Q3/Q4 单次触发与去抖流程图

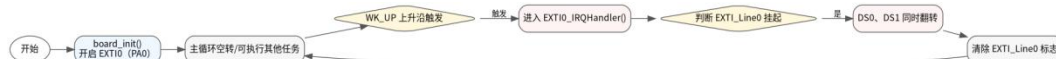


图 6-3 中断方式 Q5: WK_UP/EXTI0 流程图

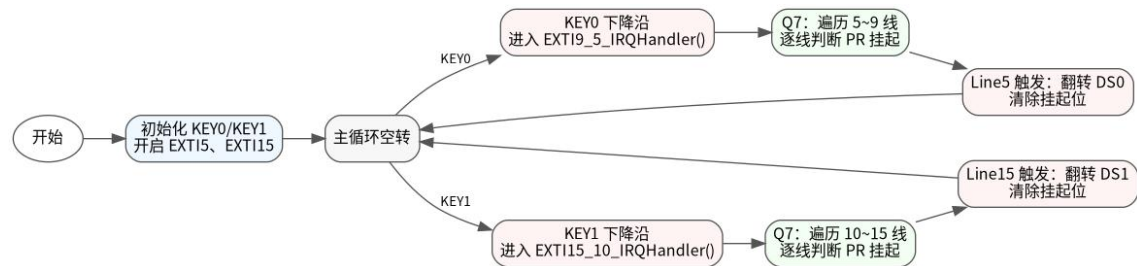


图 6-4 中断方式 Q6/Q7: KEY0/KEY1 与多线判断流程图

查询方式的核心是主循环主动读取 GPIO 电平；中断方式的核心是先完成 GPIO 到 EXTI 线的映射，再由 EXTI 挂起位判断实际触发源。Q7 的遍历处理方式比 Q6 更通用，后续即使同一中断入口下再增加其他 EXTI 线，也只需要在 handle_grouped_exti_line() 中补充对应动作。

七、程序调试及结果分析

7.1 调试结果

子工程	操作	预期/观察现象	结果分析
Q1	按下/松开 KEY0	KEY0 按下时 DS0 亮, 松开时 DS0 灭, DS1 保持灭。	主循环直接读取 PC5, 按键电平实时映射到 PA8。
Q2	按下 KEY1 或 WK_UP	KEY1 按下时 DS1 亮; WK_UP 按下时 DS0 亮、DS1 灭; WK_UP 松开后 DS0 灭、DS1 跟随 KEY1。	WK_UP 分支优先执行, 覆盖 KEY1 对 DS1 的控制。
Q3	分别短按 KEY0、KEY1、WK_UP	KEY0 每按一次翻转 DS0; KEY1 每按一次翻转 DS1; WK_UP 每按一次同时翻转 DS0、DS1。	通过上一拍状态判断新按下事件, 避免长按连续翻转。
Q4	快速按下并松开按键	稳定单击才触发翻转, 抖动期间不重复触发。	20ms 延时二次确认, 并等待释放后再允许下一次触发。
Q5	连续按 WK_UP	第一次 DS0、DS1 同时亮, 第二次同时灭, 依次翻转。	PA0 上升沿触发 EXTI0, 中断中统一翻转两个 LED。
Q6	连续按 KEY0/KEY1	KEY0 每按一次翻转 DS0; KEY1 每按一次翻转 DS1。	EXTI5 与 EXTI15 分属不同中断入口, 分别处理。
Q7	连续按 KEY0/KEY1	现象与 Q6 一致, 但中断入口按组遍历多条线。	适合 EXTI9_5、EXTI15_10 多线共用入口的通用写法。

7.2 调试过程中重点问题及处理方法

(1) KEY1 无法正常作为普通按键输入。问题原因是 PA15 默认与 JTAG 的 JTDI 复用, 若不释放该引脚, GPIO 读取可能异常。处理方法是在 `board_gpio_init()` 中调用 `GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE)`, 关闭 JTAG、保留 SWD 调试。

(2) LED 亮灭逻辑与直觉相反。DS0、DS1 均为低电平点亮, 因此上层程序不能直接把“输出高电平”理解为“亮”。处理方法是在 `board_led0_set()` 和 `board_led1_set()` 中统一封装: `on=true` 时调用 `GPIO_ResetBits()`, `on=false` 时调用 `GPIO_SetBits()`。

(3) 查询方式下长按会产生多次翻转。Q3 通过上一拍状态实现边沿检测; Q4 进一步加入 20ms 按下确认、等待释放和释放后确认, 使一次机械按键动作只产生一次稳定事件。

(4) EXTI9_5 与 EXTI15_10 入口可能对应多条中断线。Q7 使用 `for` 循环逐线检查 `EXTI_GetITStatus()`, 只处理已经挂起的线, 并在处理后清除对应挂起位, 避免遗漏或误清除。

八、思考题

8.1 GPIO 处理的查询方式和中断方式有何特点? 以按键处理为例说明 EXTI 方式有何优点,

EXTI 与查询方式比较需要增加哪些硬件?

查询方式的特点是程序结构简单, 逻辑直观, 主循环中反复读取 GPIO 输入即可完成判断。它适合任务较少、响应实时性要求不高的场景。缺点是 CPU 必须持续扫描按键, 响应时间取决于扫描周期; 如果主循环中还有较长延时或其他任务, 可能造成按键响应迟钝甚至漏检。

中断方式的特点是由硬件边沿检测触发中断, 平时 CPU 不需要不断查询按键状态, 主程序可以执行其他任务或处于空闲状态。当按键出现有效边沿时, EXTI 立即挂起中断并进入对应中断服务函数, 因此响应更及时, CPU 利用率更高, 也更适合多任务程序。

以按键处理为例, EXTI 方式的优点是: 按键未动作时不占用主循环扫描时间; 按键动作发生时能快速响应; 主程序结构更清晰, 可以把“事件触发”与“后台任务”分开。与查询方式相比, 一般不需要增加外部硬件器件, 但需要使用 STM32 内部的 AFIO/GPIO 到 EXTI 线映射、EXTI 边沿检测与挂起寄存器、NVIC 中断控制器和中断服务函数。

8.2 多个中断线共用一个中断源时, 若配置 `EXTI_Trigger_Rising_Falling`, 应如何区分上升沿还是下降沿? STM32F103 是否提供寄存器判断边沿?

STM32F103 的 EXTI->PR 挂起寄存器只能判断“哪一条 EXTI 线触发了中断”, 不能判断这次触发究竟来自上升沿还是下降沿。RTSR 和 FTSR 只用于配置允许上升沿、下降沿触发, 并不是触发结果状态寄存器。因此, 当 `EXTI_Trigger_Rising_Falling` 同时打开上升沿和下降沿时, STM32F103 没有直接提供一个寄存器告诉软件本次中断属于上升沿还是下降沿。

常用解决方法是在进入中断后立即读取对应 GPIO 的当前电平: 若当前电平为高, 通常可判断为刚发生上升沿; 若当前电平为低, 通常可判断为刚发生下降沿。对于机械按键, 还应加入短延时或状态机去抖, 二次读取电平后再确认边沿类型, 避免抖动导致误判。

如果需要更精确地区分边沿, 可采用两种改进方式: 一是只使能一个方向的边沿, 在中断中根据当前状态动态切换下一次触发边沿; 二是使用定时器输入捕获等外设记录边沿事件。对本实验按键场景而言, 读取 GPIO 当前电平并结合去抖处理已经能够满足判断需求。